Lab Sheet-04

Data Structure I (BTCS209P)

B.Tech III Semester (Session 2026-2027)

Objective:

To understand and implement the fundamental Linked List data structure using C++, and to analyze its representation, operations, algorithms, time complexity, space complexity, and practical applications.

Requirements:

C++ Compiler / IDE

VS Code, Code::Blocks, Dev-C++, or any suitable C++ environment

Basic knowledge of pointers, structures/classes, functions, loops, and conditional statements

Concepts Covered:

1. Linked List and Dynamic Memory Allocation
2. Node Representation
3. Singly Linked List
4. Insertion at Beginning, End, and a Given Position
5. Deletion from Beginning, End, and a Given Position
6. Traversal / Display
7. Searching
8. Counting Nodes
9. Linked List Overflow and Empty List
10. Reversing a Linked List
11. Applications of Linked List

1. Linked List

A Linked List is a linear dynamic data structure consisting of a sequence of nodes. Each node contains data and a pointer (link) to the next node. Unlike arrays, Linked Lists do not require contiguous memory locations.

2. Node Representation

A node in a Singly Linked List contains two parts: Data and a pointer to the next node.

Example:

[10 | next] → [20 | next] → [30 | NULL]

The pointer called Head stores the address of the first node.

3. Singly Linked List

In a Singly Linked List, each node points only to the next node. The last node points to NULL.

Head → [10 | •] → [20 | •] → [30 | NULL]

4. Insertion at Beginning

A new node is inserted before the current first node. The Head is then updated to point to the new node.

Algorithm:

INSERT_BEGIN(value)

1. Create a new node.
2. Store value in newNode->data.
3. Set newNode->next = Head.
4. Set Head = newNode.
5. Stop.

Time and Space Complexity:

Time: O(1), Auxiliary Space: O(1) excluding the newly allocated node.

Properties:

Insertion at the beginning is a constant-time operation because no traversal is required.

5. Insertion at End

A new node is added after the current last node.

Algorithm:

INSERT_END(value)

1. Create a new node and store value.
2. Set newNode->next = NULL.
3. If Head == NULL, set Head = newNode and stop.
4. Traverse from Head until the last node.
5. Set lastNode->next = newNode.
6. Stop.

Time and Space Complexity:

Time: O(n) with only a Head pointer, Auxiliary Space: O(1) excluding the new node.

Properties:

The complete list may need to be traversed to reach the last node.

6. Insertion at a Given Position

A new node is inserted at a specified valid position in the Linked List.

Algorithm:

INSERT_POSITION(value, position)

1. If position is the first position, perform insertion at beginning.
2. Create a new node.
3. Traverse to the node immediately before the required position.

4. Set newNode->next = current->next.

5. Set current->next = newNode.

6. Stop.

Time and Space Complexity:

Time: O(n) in the worst case, Auxiliary Space: O(1) excluding the new node.

7. Deletion from Beginning

The first node is removed and Head is updated to the second node.

Algorithm:

DELETE_BEGIN()

1. Check if Head == NULL.

2. If yes, display "List is Empty" and stop.

3. Store Head in a temporary pointer.

4. Set Head = Head->next.

5. Delete the temporary node.

6. Stop.

Time and Space Complexity:

Time: O(1), Auxiliary Space: O(1).

8. Deletion from End

The last node is removed from the Linked List.

Algorithm:

DELETE_END()

1. Check if Head == NULL.

2. If yes, display "List is Empty" and stop.

3. If only one node exists, delete it and set Head = NULL.

4. Otherwise, traverse to the second-last node.

5. Delete the last node and set secondLast->next = NULL.

6. Stop.

Time and Space Complexity:

Time: O(n) with only a Head pointer, Auxiliary Space: O(1).

9. Traversal / Display

Traversal visits every node from Head to NULL and displays the stored data.

Algorithm:

DISPLAY()

1. Set current = Head.
2. If Head == NULL, display "List is Empty".
3. Otherwise, repeat while current != NULL:
4. Display current->data.
5. Set current = current->next.
6. Stop.

Time and Space Complexity:

Time: O(n), Auxiliary Space: O(1) for iterative traversal.

10. Searching

Searching checks each node sequentially until the required element is found or the end of the list is reached.

Time and Space Complexity:

Time: O(n) worst case, Auxiliary Space: O(1) for iterative search.

11. Complete C++ Singly Linked List Implementation

```
#include <iostream>
using namespace std;

class LinkedList {
    struct Node {
        int data;
        Node* next;
        Node(int value) {
            data = value;
            next = NULL;
        }
    };

    Node* head;

public:
    LinkedList() {
        head = NULL;
    }

    void insertBeginning(int value) {
        Node* newNode = new Node(value);
        newNode->next = head;
        head = newNode;
    }

    void insertEnd(int value) {
        Node* newNode = new Node(value);

        if (head == NULL) {
            head = newNode;
            return;
```

```
    }

    Node* current = head;
    while (current->next != NULL)
        current = current->next;

    current->next = newNode;
}

void deleteBeginning() {
    if (head == NULL) {
        cout << "List is Empty!" << endl;
        return;
    }

    Node* temp = head;
    head = head->next;
    delete temp;
}

void deleteEnd() {
    if (head == NULL) {
        cout << "List is Empty!" << endl;
        return;
    }

    if (head->next == NULL) {
        delete head;
        head = NULL;
        return;
    }

    Node* current = head;
    while (current->next->next != NULL)
        current = current->next;

    delete current->next;
    current->next = NULL;
}

void search(int value) {
    Node* current = head;
    int position = 1;

    while (current != NULL) {
        if (current->data == value) {
            cout << "Element found at position "
                 << position << endl;
            return;
        }
        current = current->next;
        position++;
    }

    cout << "Element not found." << endl;
}
```

```
    void display() {
        if (head == NULL) {
            cout << "List is Empty!" << endl;
            return;
        }

        Node* current = head;
        cout << "Linked List: ";

        while (current != NULL) {
            cout << current->data << " -> ";
            current = current->next;
        }

        cout << "NULL" << endl;
    }
};

int main() {
    LinkedList list;
    int choice, value;

    do {
        cout << "\n--- LINKED LIST MENU ---\n";
        cout << "1. Insert at Beginning\n";
        cout << "2. Insert at End\n";
        cout << "3. Delete from Beginning\n";
        cout << "4. Delete from End\n";
        cout << "5. Search\n";
        cout << "6. Display\n";
        cout << "7. Exit\n";
        cout << "Enter choice: ";
        cin >> choice;

        switch (choice) {
        case 1:
            cout << "Enter value: ";
            cin >> value;
            list.insertBeginning(value);
            break;

        case 2:
            cout << "Enter value: ";
            cin >> value;
            list.insertEnd(value);
            break;

        case 3:
            list.deleteBeginning();
            break;

        case 4:
            list.deleteEnd();
            break;
```

```
case 5:
            cout << "Enter value to search: ";
            cin >> value;
            list.search(value);
            break;

        case 6:
            list.display();
            break;

        case 7:
            cout << "Program terminated." << endl;
            break;

        default:
            cout << "Invalid choice!" << endl;
        }

    } while (choice != 7);

    return 0;
}
```

12. Time and Space Complexity

Insert at Beginning: O(1) | Insert at End: O(n) | Delete from Beginning: O(1) | Delete from End: O(n) | Search: O(n) | Display: O(n) | Space: O(n)

13. Linked List Overflow and Empty List

A Linked List uses dynamic memory allocation, so it does not have a fixed-size overflow condition like an array. In practice, insertion can fail if sufficient memory cannot be allocated. An Empty List occurs when Head == NULL.

14. Applications of Linked List

1. Dynamic memory management
2. Implementation of stacks and queues
3. Polynomial representation
4. Graph adjacency lists
5. Hash table chaining
6. Undo/Redo and navigation structures
7. Memory-efficient insertion and deletion where positions are known

15. Lab Exercises

1. Write a C++ program to create and display a Singly Linked List.
2. Write a C++ program to insert a node at the beginning of a Linked List.
3. Write a C++ program to insert a node at the end of a Linked List.
4. Write a C++ program to insert a node at a given position.
5. Write a C++ program to delete a node from the beginning.
6. Write a C++ program to delete a node from the end.

7. Write a C++ program to delete a node from a given position.
8. Write a C++ program to search for an element in a Singly Linked List.
9. Write a C++ program to count the total number of nodes in a Linked List.
10. Write a C++ program to find the maximum and minimum element in a Linked List.
11. Write a C++ program to reverse a Singly Linked List.
12. Write a C++ program to sort the elements of a Linked List in ascending order.
13. Write a menu-driven C++ program implementing insertion, deletion, searching and display operations.
14. Write a C++ program to implement a Doubly Linked List and perform insertion and deletion operations.
15. Write a C++ program to implement a Circular Linked List and perform traversal and insertion operations.

Practice / Analysis Questions

1. What is a Linked List and how is it different from an array?
2. What is a node in a Linked List?
3. What is the role of the Head pointer?
4. What is dynamic memory allocation?
5. What is the difference between Singly and Doubly Linked Lists?
6. What is the time complexity of insertion at the beginning?
7. Why does insertion at the end take O(n) with only a Head pointer?
8. What is the time complexity of searching in a Linked List?
9. What happens when Head == NULL?
10. What are the advantages and disadvantages of Linked Lists?
11. How can a Linked List be used to implement a Stack?
12. How can a Linked List be used to implement a Queue?
13. What is a Circular Linked List?
14. What is the difference between Linear and Circular Linked Lists?
15. Compare Array and Linked List in terms of memory allocation, access, insertion and deletion.

Instructions & Guidelines

- Write a separate C++ program for each assigned exercise.
- Use structures/classes and functions wherever appropriate to make programs modular and easy to understand.
- Use dynamic memory allocation for Linked List nodes.
- Release dynamically allocated memory using delete where appropriate.
- Test each program with empty, single-node, and multiple-node lists.
- Display input and output clearly.
- Add comments to important parts of the program.
- Mention the time and space complexity of each implemented operation.
- Do not use STL list functions for implementing the basic Linked List operations in the assigned exercises.

Submission

Submit the C++ source code files for the assigned tasks along with a brief report containing the objective, concept/theory, algorithm, program, sample input/output, result, and time and space complexity analysis of the implemented Linked List operations.